Lab 9.1 - Consolidation Lab: Build a 2-VM Environment from Scratch

KVM Virtualisation on Oracle Linux - Day 2, Module 9

Overview

This is a closed-book, CLI-only lab. Rather than introducing new concepts, it asks you to combine everything from Modules 2 through 8 into a single build: two new VMs, sharing a dedicated storage pool and a bridged network, one of them carrying a second data disk, both protected by a snapshot, with a deliberate fault-and-restore cycle to close it out. No virt-manager, no shortcuts - if a step isn't clear, the answer is almost always in one of today's earlier labs.

By the end, you will have app-vm and db-vm running side by side: two independent virtual machines built the same way you built ol-lab-01, using only the tools covered today.

Learning Objective

By the end of this lab, you will be able to:

- Build a complete 2-VM environment from a bare hypervisor using only virsh and virt-install
- Provision dedicated storage and networking for a multi-VM environment
- Apply snapshot protection before a risky change, and understand what a snapshot does and doesn't protect
- Produce a clear, documented final state of a KVM environment you built yourself

Step by Step Guide

Prerequisite: Connect to Your Lab VM

Your OCI lab host was provisioned via the Terraform workflow covered earlier in the programme. Connect to it and confirm the host is in the state left by today's earlier labs before starting this build.

1. Connect to your lab host over SSH:

ssh oracle@<your-lab-instance-ip>

Expected result: A shell prompt on your OCI lab instance.

2. Confirm libvirtd is running and both networks from earlier today are active:

```
systemctl status libvirtd
virsh net-list --all
```

Expected result: libvirtd active (running); both default and br0-network listed as active, autostart yes.

Step 1: Create a Dedicated Storage Pool

Build a new pool, separate from both default and labpool, to hold this lab's VM disks.

3. Define, build, start, and autostart the pool:

```
virsh pool-define-as consolidation-pool dir \
  --target /var/lib/libvirt/images/consolidation-pool
virsh pool-build consolidation-pool
virsh pool-start consolidation-pool
virsh pool-autostart consolidation-pool
```

4. Confirm it's active:

```
virsh pool-list --all
```

Expected result: consolidation-pool listed as active, alongside default and labpool from earlier labs.

Step 2: Create Two VMs with virt-install

Build app-vm and db-vm the same way ol-lab-01 was built in Lab 3.1, both attached to the bridged network so they can reach each other and the wider lab network directly.

5. Create app-vm:

```
virt-install \
  --name app-vm --memory 1024 --vcpus 1 \
  --disk pool=consolidation-pool,size=15,format=qcow2 \
  --cdrom /var/lib/libvirt/isos/OracleLinux-9-x86_64-dvd.iso \
  --os-variant ol9 --network network=br0-network \
  --graphics vnc,listen=0.0.0.0 --noautoconsole
```

6. Create db-vm the same way:

```
virt-install \
  --name db-vm --memory 1024 --vcpus 1 \
  --disk pool=consolidation-pool,size=15,format=qcow2 \
```

```
--cdrom /var/lib/libvirt/isos/OracleLinux-9-x86_64-dvd.iso \
--os-variant ol9 --network network=br0-network \
--graphics vnc,listen=0.0.0.0 --noautoconsole
```

Expected result: Both commands complete with domain creation confirmed, and both VMs appear running in virsh list --all.

7. Complete the guided OS installation for each VM via its console, exactly as in Lab 3.1:

```
virt-viewer --connect qemu:///system app-vm
virt-viewer --connect qemu:///system db-vm
```

Note: Adjust the --cdrom path and --os-variant to match your lab environment's actual ISO location and Oracle Linux version, as in Lab 3.1.

Step 3: Confirm Both VMs Are Up

8. Once both installs are complete and the VMs have rebooted into their installed OS, confirm both are running with network addresses on the bridged segment:

```
virsh list --all
virsh domifaddr app-vm
virsh domifaddr db-vm
```

Expected result: Both VMs listed as running, each with an address in the same segment as the host's br0 bridge.

Step 4: Take an Initial Snapshot of Both VMs

With both VMs confirmed working, capture this state as a baseline before making any further changes.

```
virsh snapshot-create-as app-vm baseline "Base OS install confirmed"
virsh snapshot-create-as db-vm baseline "Base OS install confirmed"
```

Expected result: Both commands report the snapshot created; virsh snapshot-list app-vm and virsh snapshot-list db-vm each show one entry.

Step 5: Attach a Second Data Disk to db-vm

9. Create a data volume in the new pool:

```
virsh vol-create-as consolidation-pool db-vm-data.qcow2 5G --format qcow2
```

10. Attach it to db-vm, live and persistently:

```
virsh attach-disk db-vm \
```

/var/lib/libvirt/images/consolidation-pool/db-vm-data.qcow2 \

vdb --subdriver qcow2 --targetbus virtio --live --config

11. Inside db-vm's console, partition, format, and mount it:

sudo parted /dev/vdb --script mklabel gpt mkpart primary ext4 0% 100%

sudo mkfs.ext4 /dev/vdb1

sudo mkdir /data && sudo mount /dev/vdb1 /data

sudo touch /data/consolidation-lab-marker

Expected result: df -h /data shows the new disk mounted with roughly 5G available, and the marker file is created successfully.

Step 6: Simulate a Fault on db-vm, Then Restore

Break something on db-vm the same way Lab 6.1 did, then revert to the baseline snapshot from Step 4.

12. From db-vm's console, disable SSH access:

sudo systemctl stop sshd && sudo systemctl disable sshd

13. From the host, confirm the fault:

ssh oracle@<db-vm's address> 'echo SSH is working'

Expected result: Connection refused or timeout - the fault is confirmed.

14. Revert db-vm to its baseline snapshot:

virsh snapshot-revert db-vm baseline

Expected result: SSH access is restored, since the baseline snapshot captured db-vm before sshd was disabled.

Note: Look closely at db-vm after the revert: the baseline snapshot was taken in Step 4, before the data disk was attached in Step 5. Reverting restores the VM's full state as of that snapshot - including its device configuration - which means the second disk attached afterwards may no longer be present on the VM. Check with virsh domblklist db-vm. This is a genuine, important limitation of snapshots: they only protect what existed at the moment they were taken. A disk attached afterwards needs its own, later snapshot to be protected the same way.

15. If the data disk was detached by the revert, reattach it (it still exists in the pool - only the VM's reference to it was rolled back):

virsh attach-disk db-vm \

/var/lib/libvirt/images/consolidation-pool/db-vm-data.qcow2 \

vdb --subdriver qcow2 --targetbus virtio --live --config

inside the guest: sudo mount /dev/vdb1 /data

Step 7: Document the Final State

Close out the lab with a clear record of what you built, in the same way you'd document a real environment for a handover.

16. Capture the final state of VMs, networks, pools, and snapshots:

virsh list --all

virsh net-list --all

virsh pool-list --all

virsh snapshot-list app-vm --tree

virsh snapshot-list db-vm --tree

Expected result: app-vm and db-vm both listed as running; default, br0-network, and any earlier lab networks all active; default, labpool, and consolidation-pool all active; each VM shows its baseline snapshot.

17. Confirm the data marker on db-vm survived, proving the disk was correctly reattached if needed:

ls -l /data/consolidation-lab-marker

Expected result: The file is present, confirming db-vm's data disk is correctly attached and mounted in the environment's final state.

Outcomes

By the end of this lab, you have built a complete two-VM environment entirely from the command line: dedicated storage, shared bridged networking, a second data disk on one VM, and snapshot protection exercised through a real fault-and-restore cycle - including the important discovery that a snapshot only protects what existed at the moment it was taken. This lab has used virt-install, pool and volume management, network attachment, snapshot creation and reversion, and disk attachment together in one build, which is the same combination of skills a real KVM deployment requires day to day. This closes out the KVM Virtualisation on Oracle Linux programme.